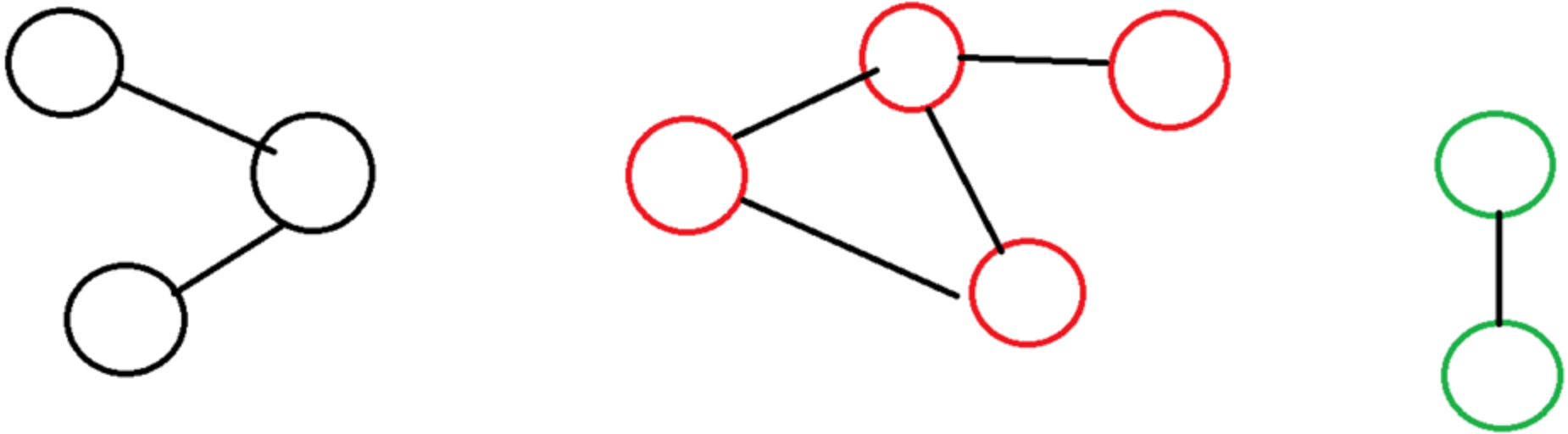
Graph Algorithms

Connected Graph
in
Undirected graphs

Connected Components: ***Undirected*** Graph

- In an undirected graph G = (V, E) two vertices u, v ∈ V are **connected** iff there is a path from u to v
- An undirected graph is **connected** if every vertex is reachable from all other vertices
- A **connected component** of G is a set C ⊆ V which has the following properties:
  - C is nonempty
  - For any u, v ∈ C: u and v are connected
  - For any u ∈ C, v ∈ V – C: u and v are not connected

Connected Components: ***Undirected*** Graph



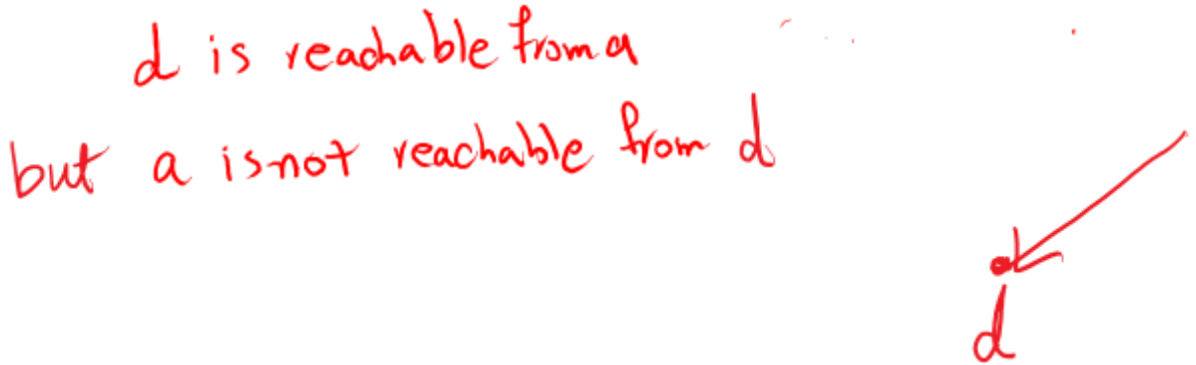
Undirected Graph

- Connected graphs
  - How to check if a graph is connected
    - Pick an arbitrary node s and check whether all vertices are reachable from s
- How to find connected components in an undirected graph?
  - Using DFS
    - DFS(G, u) finds all nodes reachable from u in the graph
- Exercise: Modify DFS to find connected components

Strongly Connected Graphs
in
Directed graphs

Strongly Connected graphs: **Directed** graph

- In a **directed graph** G
  - v is reachable from u iff there is a path from u to v.
- In an undirected graph, if there is a path from u to v, there is also a path from v to u.
- In a directed graph, it is possible for v to be reachable from u, but for u not to be reachable from v.



Strongly Connected Graphs

Input: A directed graph $G = (V, E)$

Output: Yes if G is strongly connected; no otherwise

Brute-force solutions:

- For each pair u,v check whether there is a path from u to v, v to u
  - Runtime: $O(n^2(n+m))$
- For each vertex v, whether all vertices can be reached from v
  - Runtime: $O(n(n+m))$

Strongly Connected Graphs: Observation

Lemma. G is strongly connected if and only if every vertex v is reachable from s and s is reachable from every vertex v, where s is an arbitrary vertex

- ⇒ by the definition of a strongly connected component
- ⇐ For any u,v ∈ V, we obtain a path from u to v by combining a path from u to s and a path from s to v →G is strongly connected

- How do we check whether s is reachable from every vertex $v \in V$?
  - Idea: Reverse the graph
  - Claim: Given $G = (V, E)$, we reverse the direction of all the edges to obtain $G^{\mathsf{T}} = (V, E^{\leftarrow})$. Then, there is a path from v to s in G if and only if there is a path from s to v in G^{T}. So, s is reachable from every $v \in V$ in G if and only if every $v \in V$ is reachable from s in G^{T}
- Example

Strongly Connected Graphs: Algorithm

- Check whether all vertices in G are reachable from s by one DFS
- Reverse the direction of all the edges in G to obtain G^T
- Check whether all vertices in G^T are reachable from s by one DFS
- If both yes, return “SC” graph, otherwise, return “not SC”

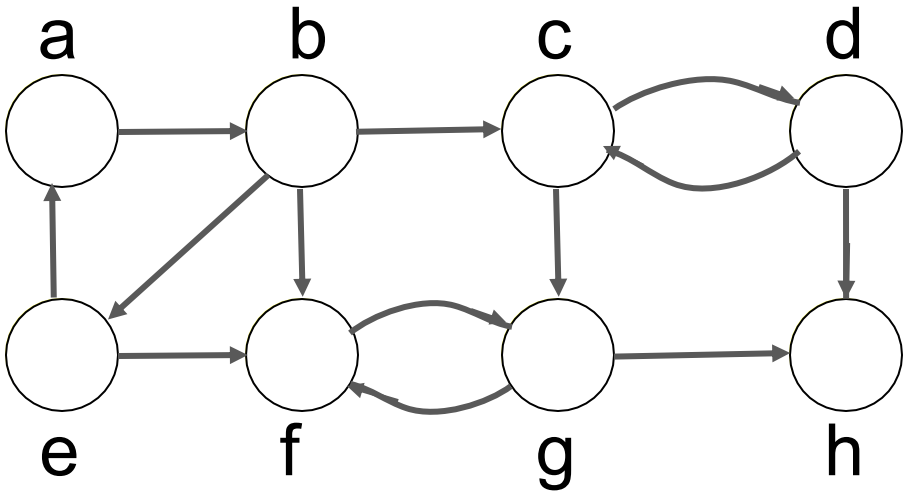
Runtime: O(m+n)

Strongly Connected Components in directed Graphs

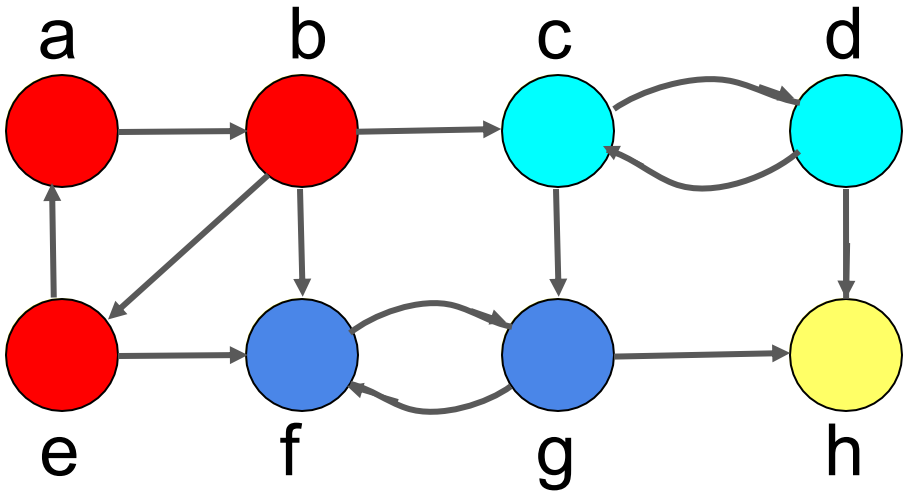
Strongly Connected Components

- Let G = (V, E) be a directed graph
- Two vertices u ∈ V and v ∈ V are **strongly connected** iff v is reachable from u and u is reachable from v
- A **directed graph is strongly connected** if and only if every pair of vertices is strongly connected.
- A **strong connected component (**or **SCC)** of G is a maximal strongly connected subgraph of G.
- A **SCC** of G is a set C ⊆ V such that:
  - C is not empty
  - For any u, v ∈ C: u and v are strongly connected
  - For any u ∈ C and v ∈ V – C: u and v are not strongly connected.

Strongly Connected Components



Strongly Connected Components



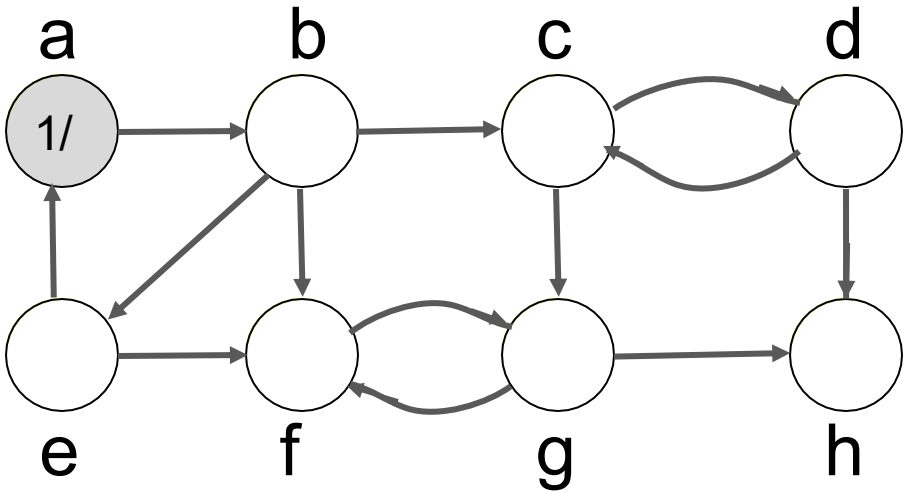
Strongly Connected Components

- Brute-force:
  - Consider all possible subset of vertices and check using the algorithm for checking graph connectivity
  - Exponential: at least we need to compute all subsets
- Better brute-force:
  - For each pair (u,v)
    - C_1 = DFS(u) to find if there are path between u and v
    - C_2 = DFS(v) to find if there are path between v and u
    - Build the SCCs accordingly
  - Runtime: $O(n^2(n + m))$
    - We need to run DFS on each pair of nodes

Strongly Connected Components: efficient algorithm

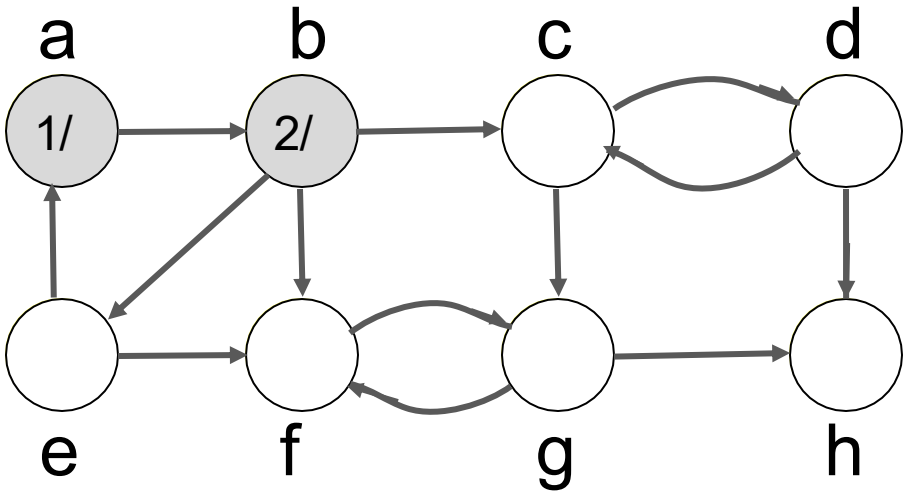
```
def StronglyConnectedComponents(G):
  1.   DFS(G)        # to compute finish time f[u] for each vertex u
  2.   Compute G^T
  3.   Call DFS(G^T) # traverse vertices in decreasing order of finish time
  4.   The SCCs are the different DFS tree in G^T
```

Strongly Connected Components: Example



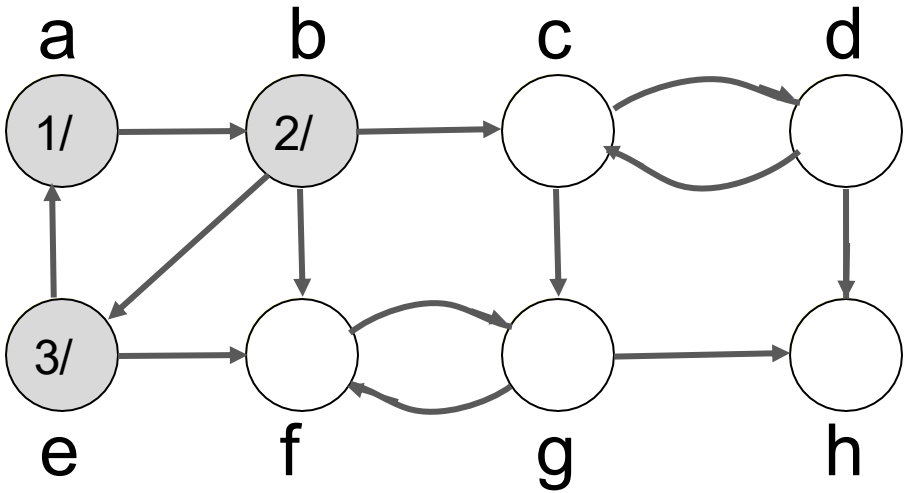
1. DFS(G) to compute finish time f[u] for each vertex u

Strongly Connected Components: Example



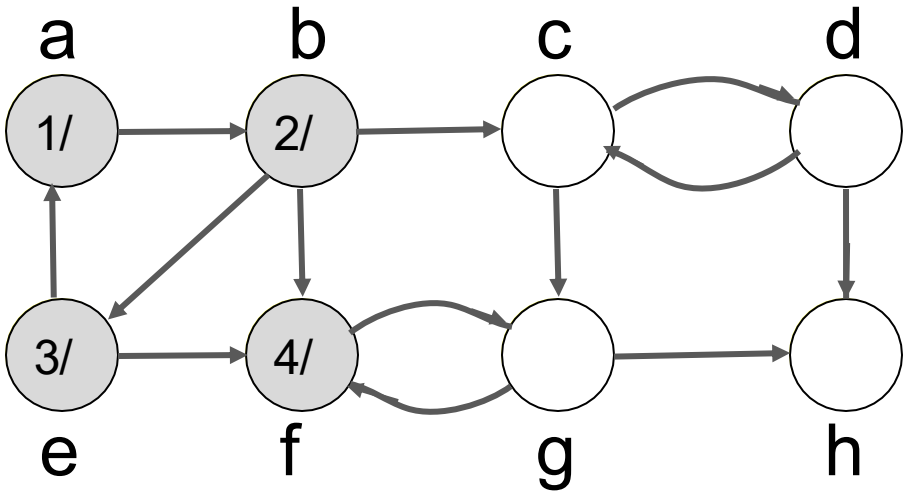
1. DFS(G) to compute finish time f[u] for each vertex u

Strongly Connected Components: Example



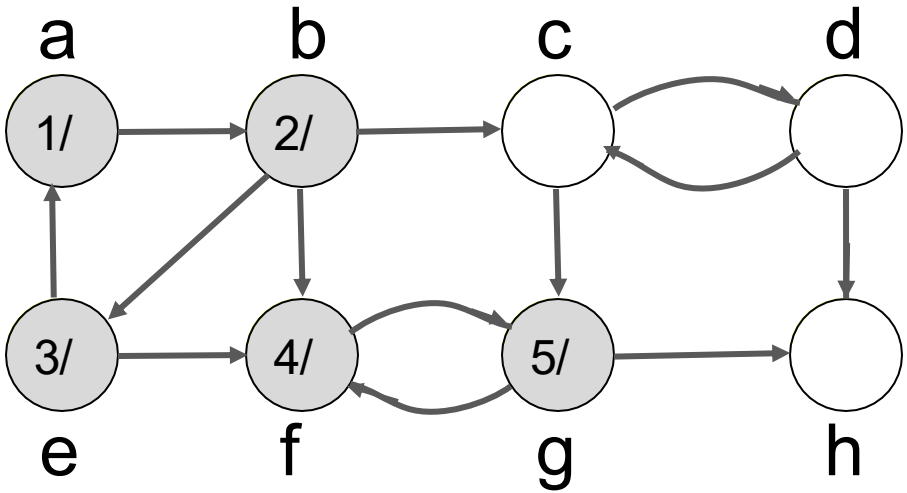
1. DFS(G) to compute finish time f[u] for each vertex u

Strongly Connected Components: Example



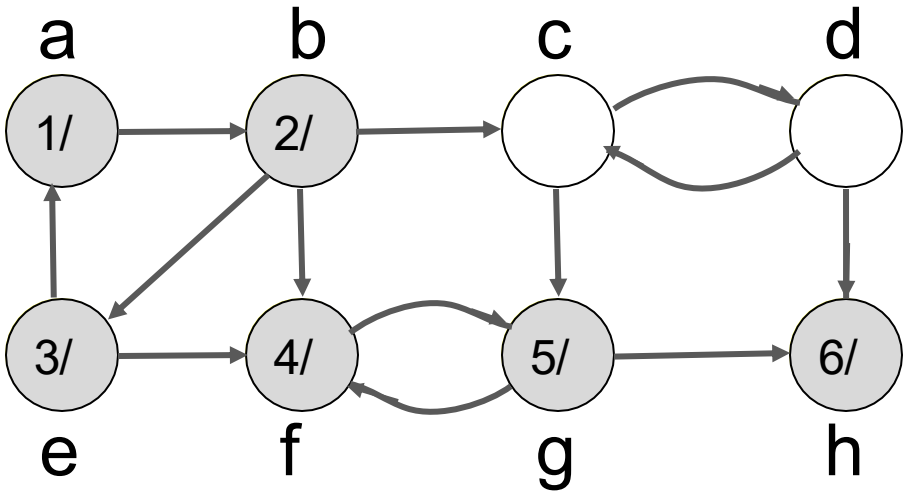
1. DFS(G) to compute finish time f[u] for each vertex u

Strongly Connected Components: Example



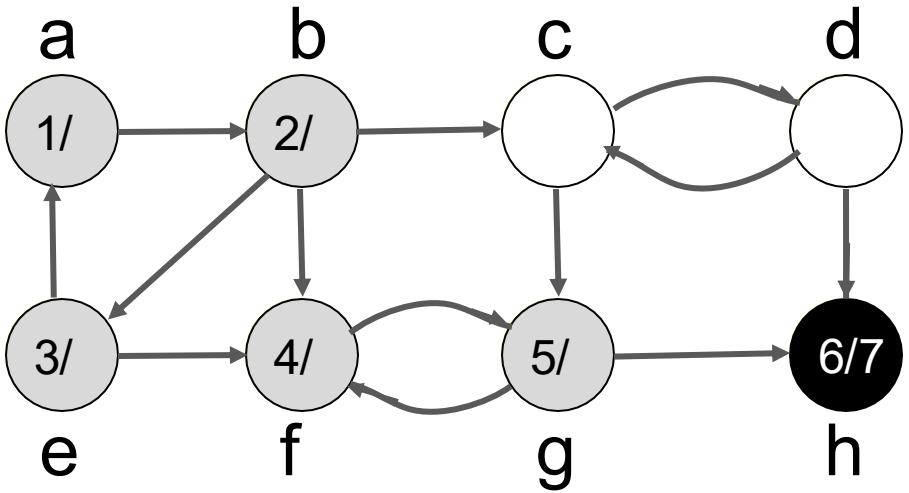
1. DFS(G) to compute finish time f[u] for each vertex u

Strongly Connected Components: Example



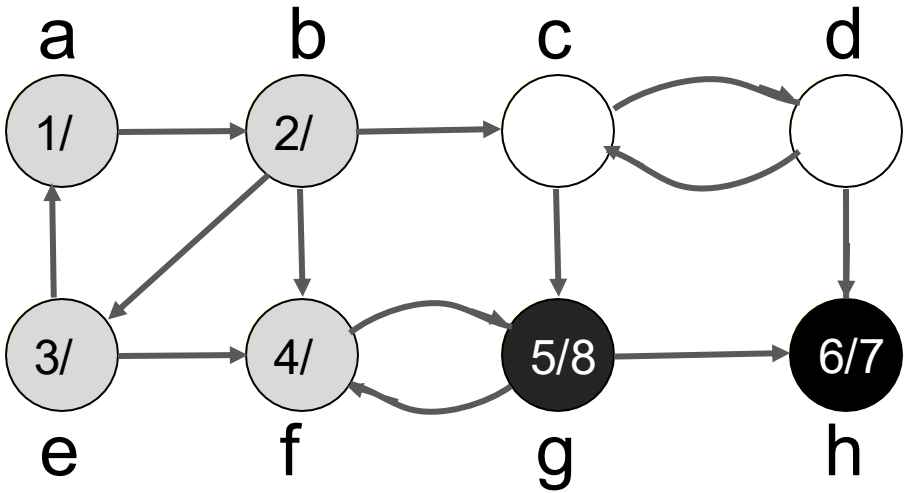
1. DFS(G) to compute finish time f[u] for each vertex u

Strongly Connected Components: Example



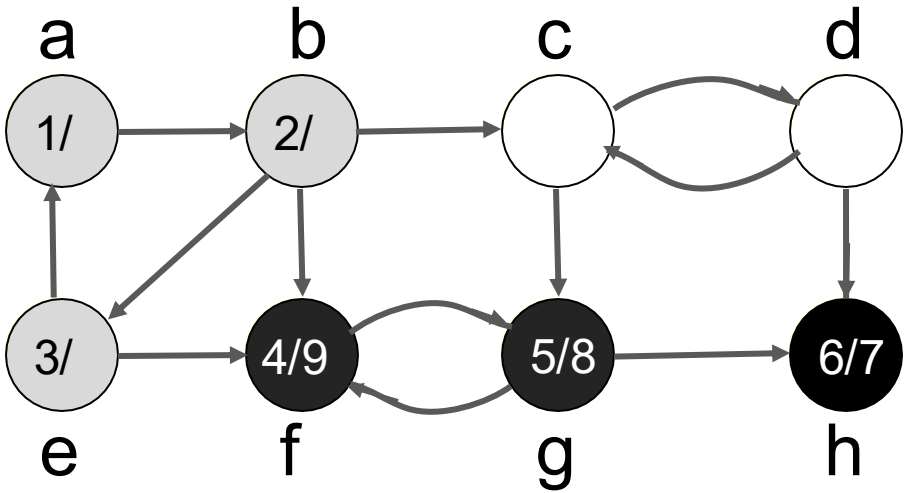
1. DFS(G) to compute finish time f[u] for each vertex u

Strongly Connected Components: Example



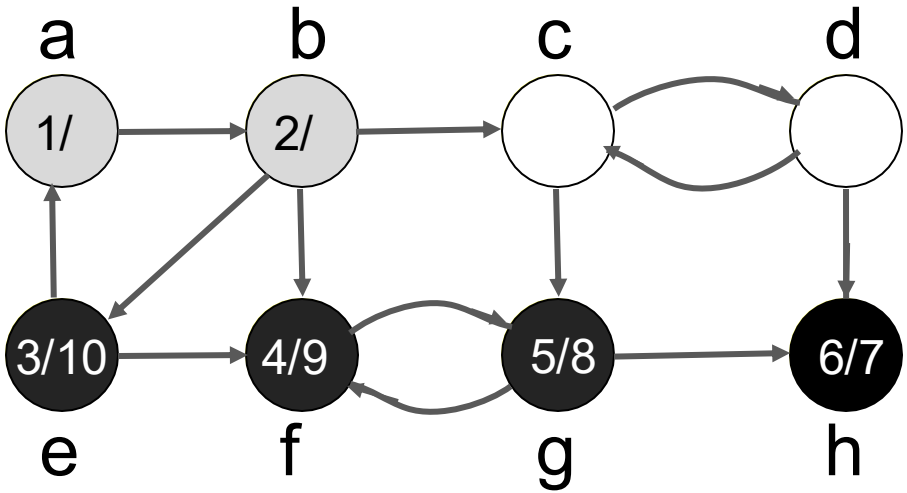
1. DFS(G) to compute finish time f[u] for each vertex u

Strongly Connected Components: Example



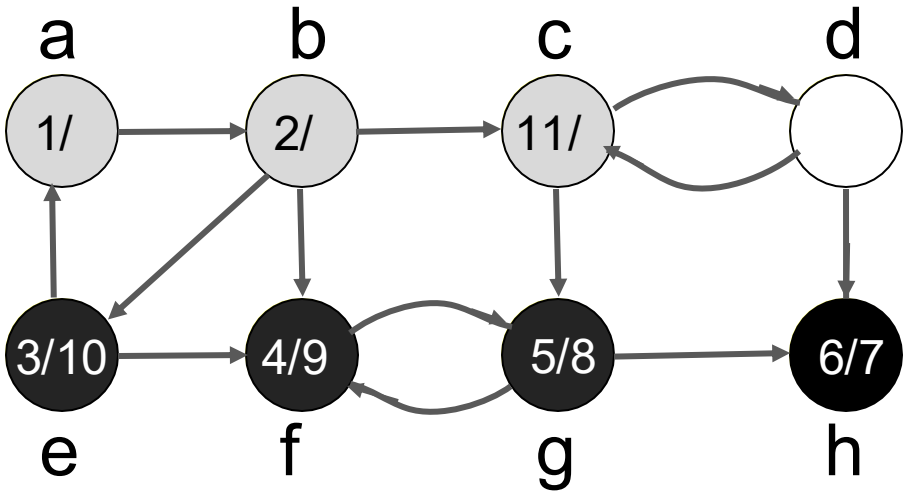
1. DFS(G) to compute finish time f[u] for each vertex u

Strongly Connected Components: Example



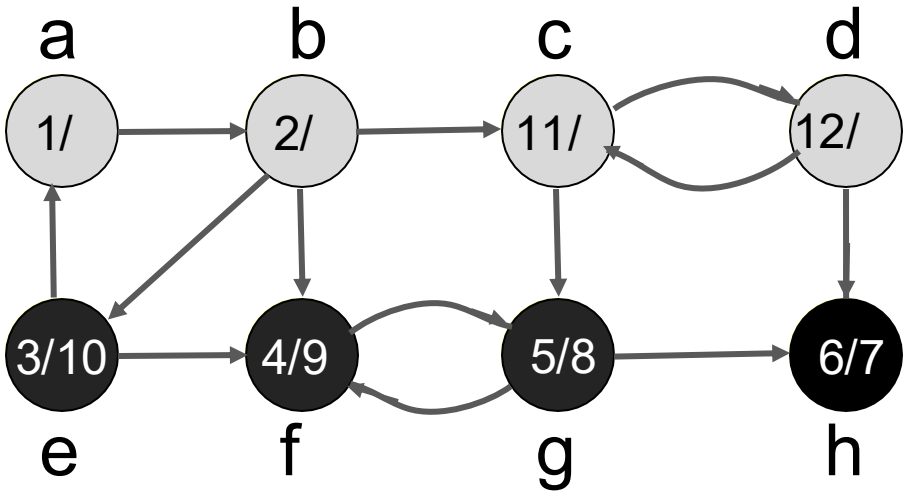
1. DFS(G) to compute finish time f[u] for each vertex u

Strongly Connected Components: Example



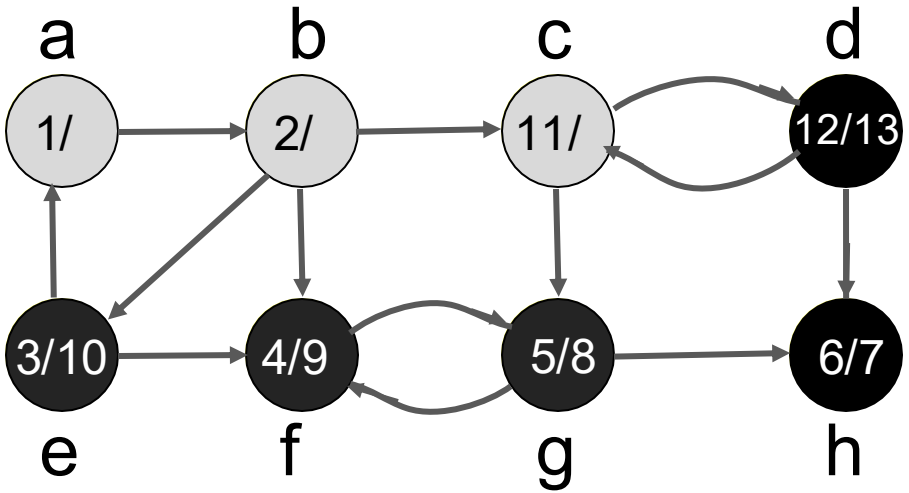
1. DFS(G) to compute finish time f[u] for each vertex u

Strongly Connected Components: Example



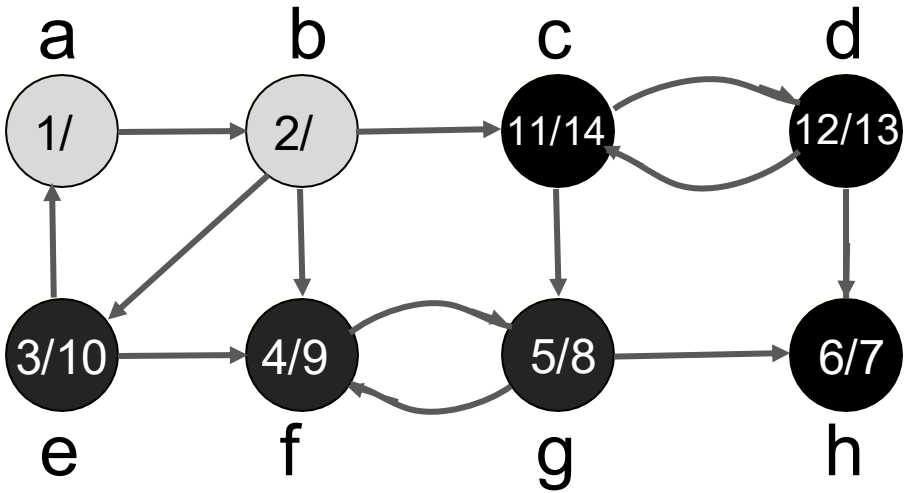
1. DFS(G) to compute finish time f[u] for each vertex u

Strongly Connected Components: Example



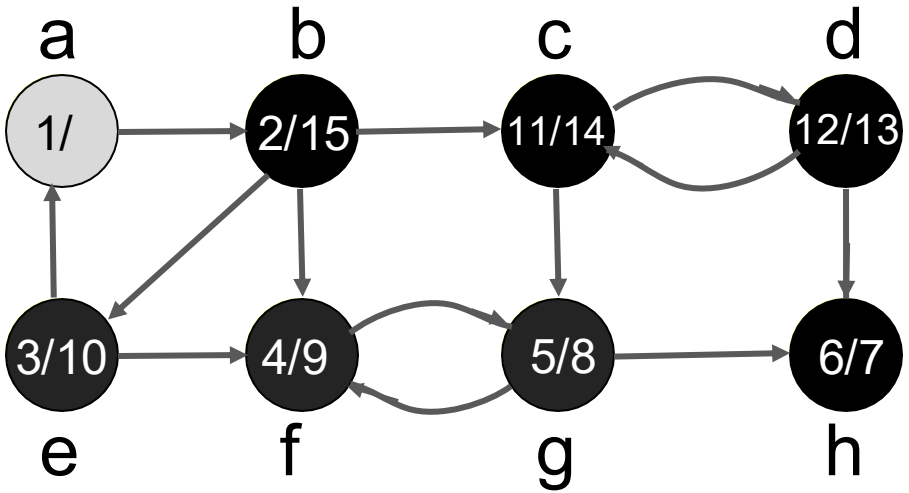
1. DFS(G) to compute finish time f[u] for each vertex u

Strongly Connected Components: Example



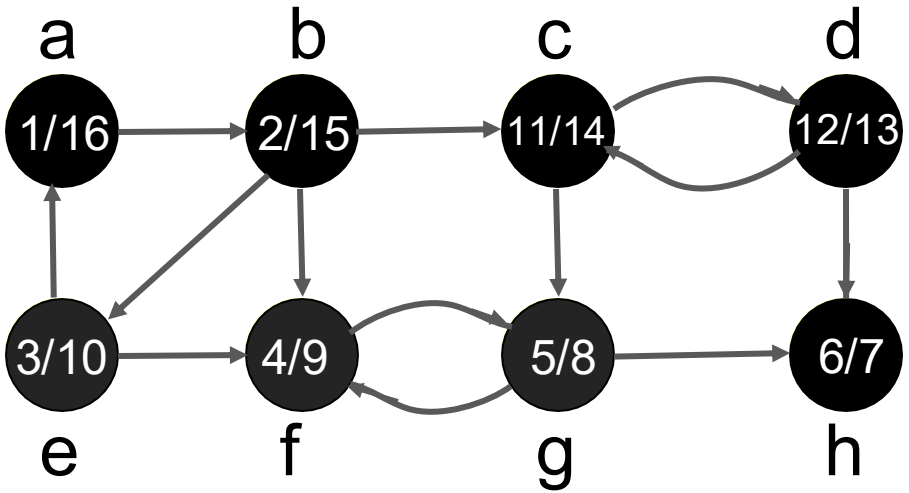
1. DFS(G) to compute finish time f[u] for each vertex u

Strongly Connected Components: Example



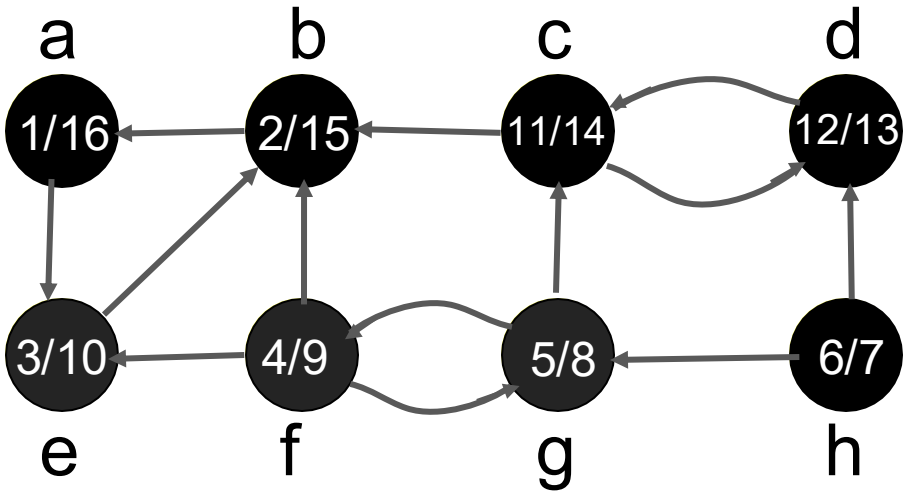
1. DFS(G) to compute finish time f[u] for each vertex u

Strongly Connected Components: Example



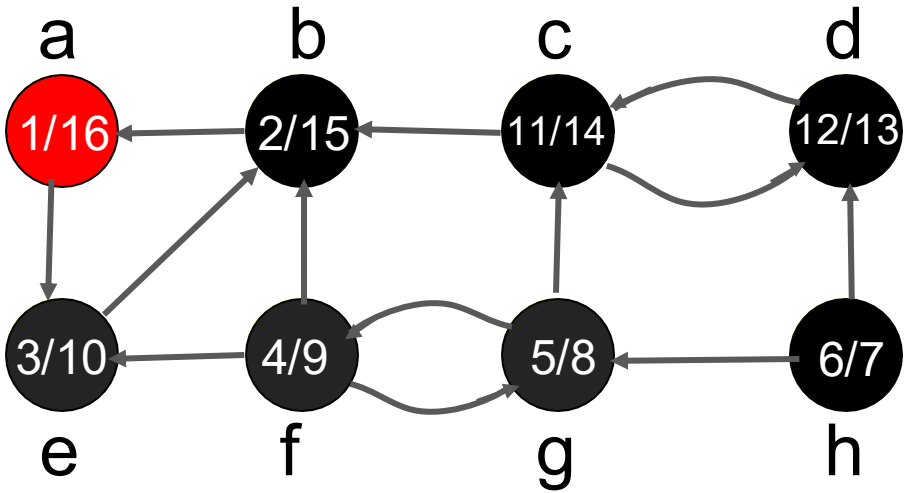
1. DFS(G) to compute finish time f[u] for each vertex u

Strongly Connected Components: Example



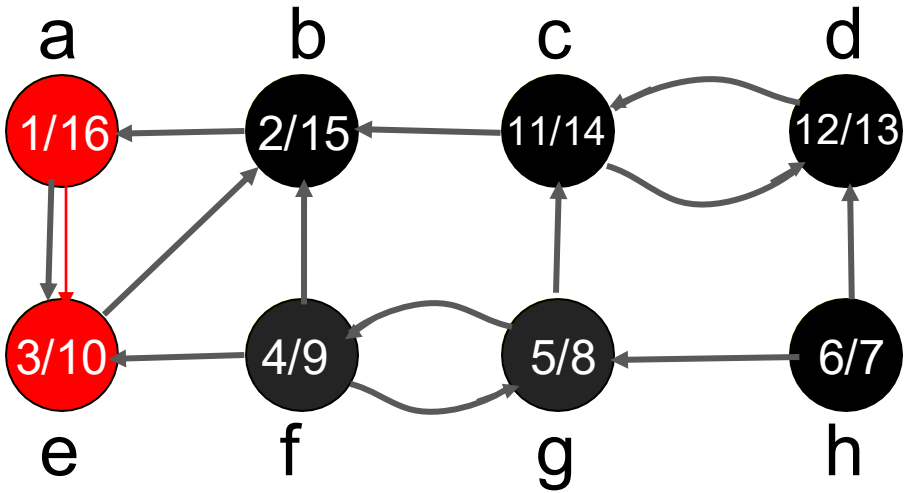
1. DFS(G) to compute finish time f[u] for each vertex u
2. Compute G^T

Strongly Connected Components: Example



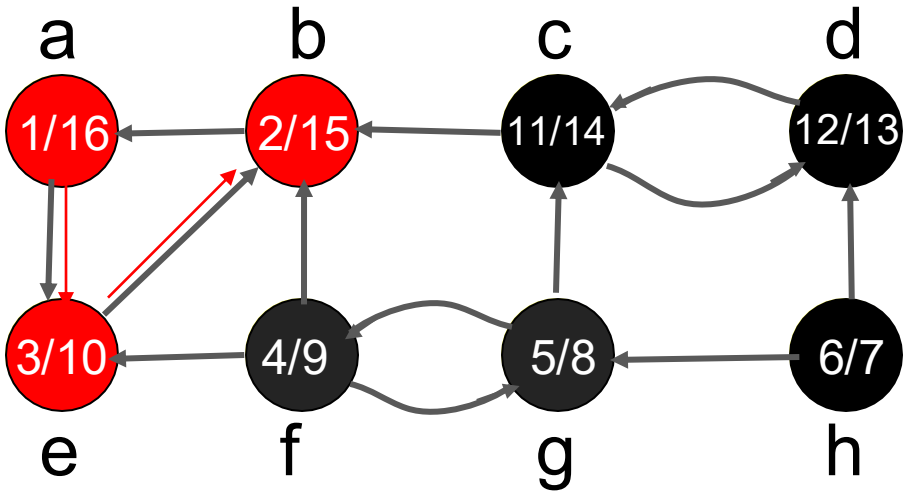
1. DFS(G) to compute finish time f[u] for each vertex u
2. Compute G^T
3. DFS(G^T) traverse vertices in decreasing order of finish time

Strongly Connected Components: Example



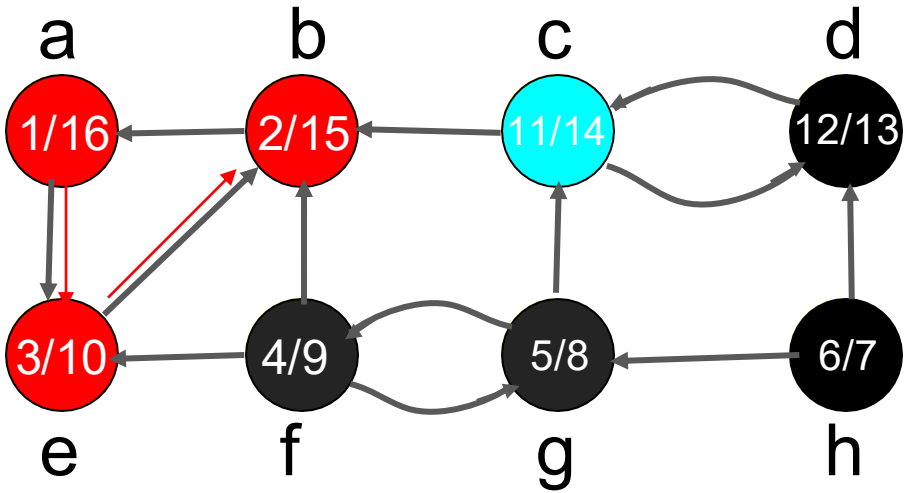
1. DFS(G) to compute finish time f[u] for each vertex u
2. Compute G^T
3. DFS(G^T) traverse vertices in decreasing order of finish time

Strongly Connected Components: Example



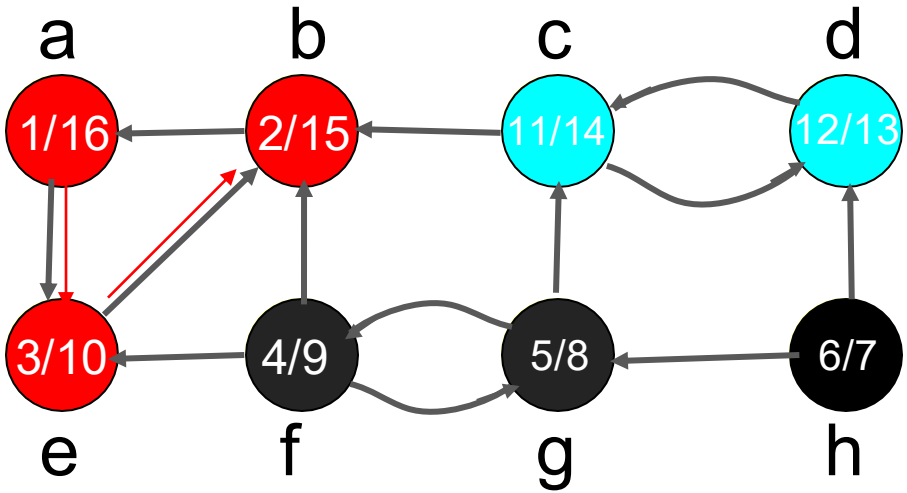
1. DFS(G) to compute finish time f[u] for each vertex u
2. Compute G^T
3. DFS(G^T) traverse vertices in decreasing order of finish time

Strongly Connected Components: Example



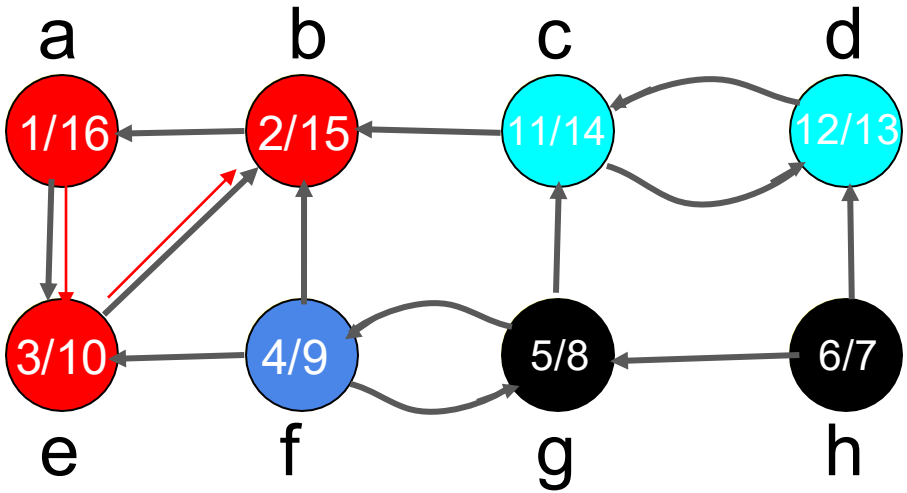
1. DFS(G) to compute finish time f[u] for each vertex u
2. Compute G^T
3. DFS(G^T) traverse vertices in decreasing order of finish time

Strongly Connected Components: Example



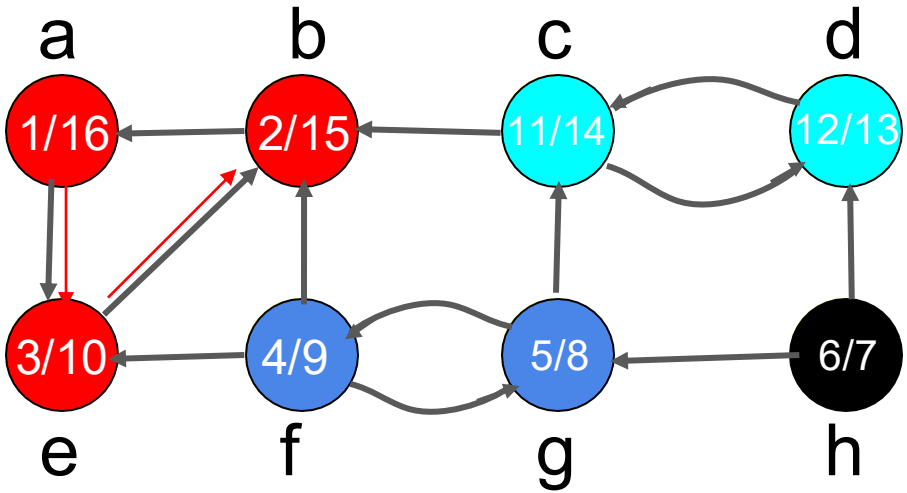
1. DFS(G) to compute finish time f[u] for each vertex u
2. Compute G^T
3. DFS(G^T) traverse vertices in decreasing order of finish time

Strongly Connected Components: Example



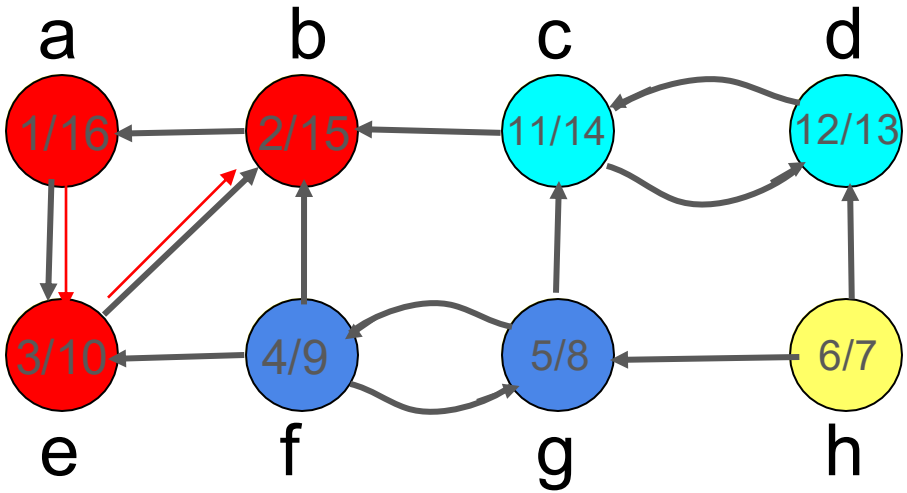
1. DFS(G) to compute finish time f[u] for each vertex u
2. Compute G^T
3. DFS(G^T) traverse vertices in decreasing order of finish time

Strongly Connected Components: Example



1. DFS(G) to compute finish time f[u] for each vertex u
2. Compute G^T
3. DFS(G^T) traverse vertices in decreasing order of finish time

Strongly Connected Components: Example



1. DFS(G) to compute finish time f[u] for each vertex u
2. Compute G^T
3. DFS(G^T) traverse vertices in decreasing order of finish time